Tentative Sol for CSL3090 AI Minor Exam 2025

Total Marks: 40 **Duration:** 120 mins

The questions have all the required information available. Read the questions carefully and donot make any unnecessary assumptions.
Use of pencil/red pen is NOT allowed. If pencil/red pen is used to answer any question, that answer will not be checked.

1. How is the Total Turing Test different from the Turing Test? What additional capabilities should an agent have to pass the Total Turing Test? [2 Marks]

   Ans:

   A computer passes the Turing test if a human interrogator, after posing some written questions, cannot tell whether the written responses come from a person or from a computer.
   The Total Turing Test includes a video signal so that the interrogator can test the subject's perceptual abilities, as well as the opportunity for the interrogator to pass physical objects "through the hatch".

   To pass the total Turing Test, the computer will additionally need
   ○ computer vision to perceive objects
   ○ robotics to manipulate objects and move about.

2. Suppose you have three jugs with capacities: JA (7 litres), JB (5 litres), and JC (2 litres). Initially, JA contains 2 litres of milk and JB contains 5 litres of milk, while JC is empty. Your goal is to achieve exactly 6 litres of milk in any one jug without throwing out or wasting any milk. Model this task as a search problem and solve it using graph-based A* search.

   Action Rules: The only permitted action is pour(sourcejug, targetjug): completely transfer milk from the source jug to the target jug until either: a) The source jug becomes empty, OR b) The target jug becomes full. Note that this action does not throw away the remaining milk in the source jug if the target jug cannot accommodate the total milk in the source jug. So if you transfer milk from a fully filled JB to an empty JC, JC will get only 2 litres, and the remaining 3 litres will remain in JB. Also, this action is not possible if the source jug is empty or the target jug is full; therefore, do not apply this action in such cases.

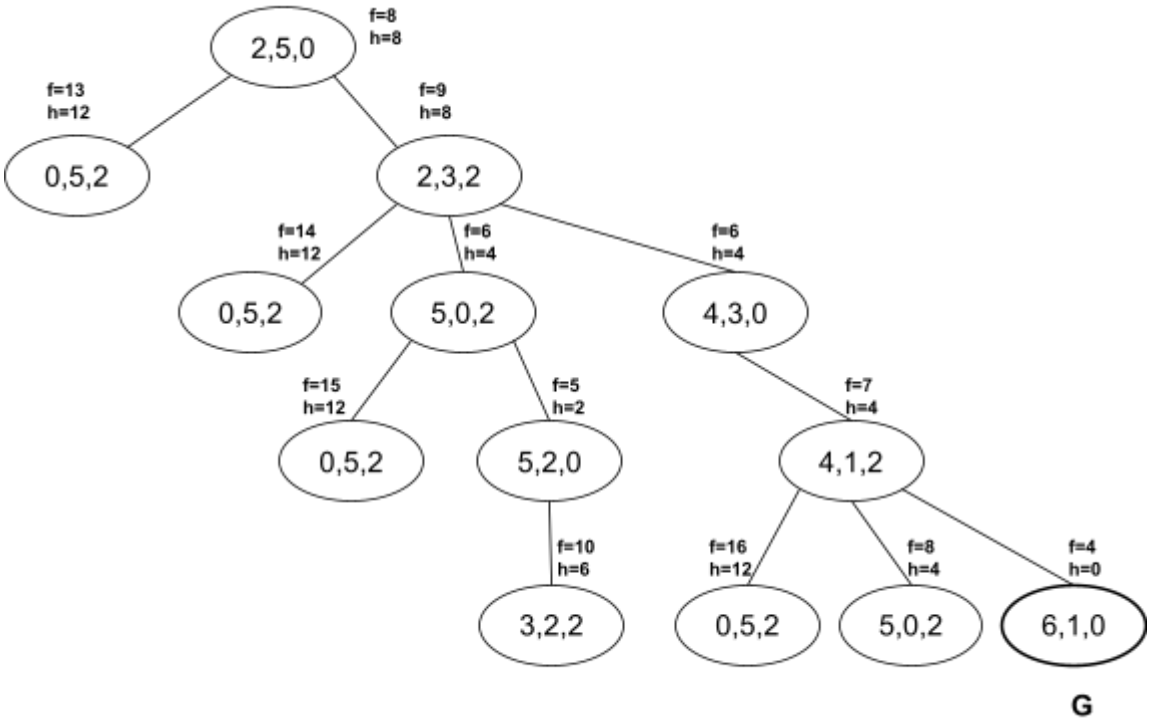
   Search Problem Formulation: The state representation is (a, b, c), where a, b, and c represent current milk amounts in JA, JB, and JC, respectively. The start state is (2,5,0). Each valid action that transitions from one state to a directly reachable neighboring state has a uniform step cost of 1. The heuristic function for any state n is as follows:

   h(n) = (-JA + abs(JB-1) + (JC+6)) where abs(.) returns the absolute value

   Additional Restrictions:
   - (7,0,0) is an invalid state and should not be shown in the tree
   - A state cannot appear as a descendant of itself in the search tree, preventing infinite cycles.

   All instructions must be followed, otherwise, marks will not be awarded.

   a. Write down all the goal state configurations. [1 Mark]
      Ans: (6,1,0), (6,0,1)
   b. Draw the search tree for A* graph search exactly at the moment when the first goal state is expanded. [6 Marks]



c. How many nodes needed to be expanded when solving this task using A* tree search (including the start state node and the first goal state to be expanded)? [2 Marks]
Ans: 7

d. Is this heuristic function admissible? Show that it is admissible or not admissible. [2 Marks]
No.
h(6,0,1)=2
h(2,5,0) = 4 + 4 = 8, however actual cost = 4

e. Suggest another nontrivial admissible heuristic function and show that it is admissible. [2 Marks]
h(n) = min(JA, JB, JC)
Consider extreme cases and show that it is admissible

3. Consider an adversarial zero-sum two-player game. We have a bucket containing 5 marbles. In this game, we have 2 players (Max and Min) who can pick either 1 or 2 marbles at a time in alternate turns. The player who draws the last marble wins. Both players want to win the game with the maximum number of marbles in their possession. The game ends when a player chooses the remaining marbles in the bucket, and the bucket becomes empty.

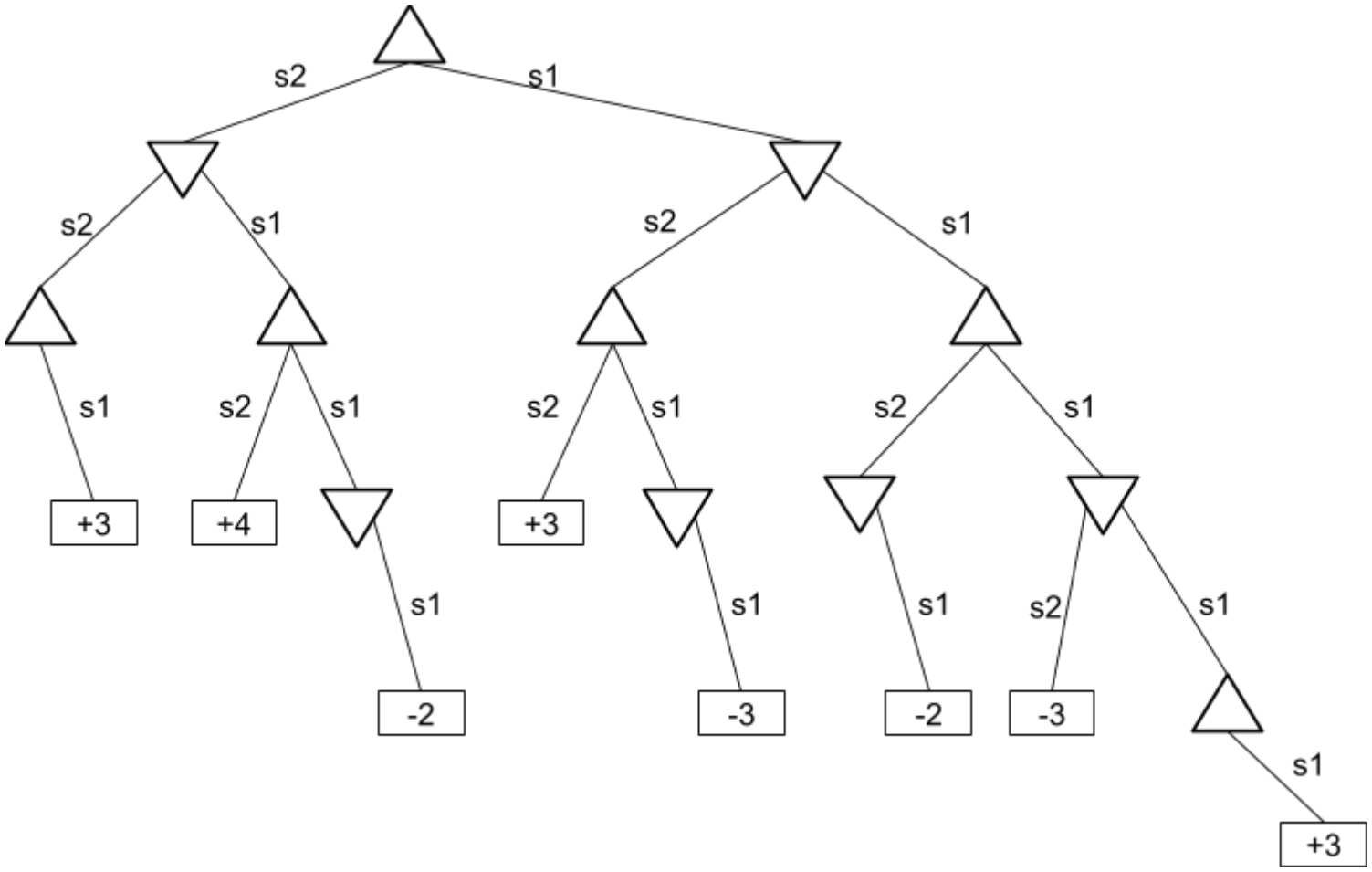
   The available actions are Pick 1 marble (s1) or 2 marbles (s2) in a turn. A player cannot pick more marbles than are currently available in the bucket.

   The terminal node is assigned a value based on which player wins the game. If Max wins: Terminal value = +X (where X = total marbles Max collected). If Min wins: Terminal value = -Y (where Y = total marbles Min collected).

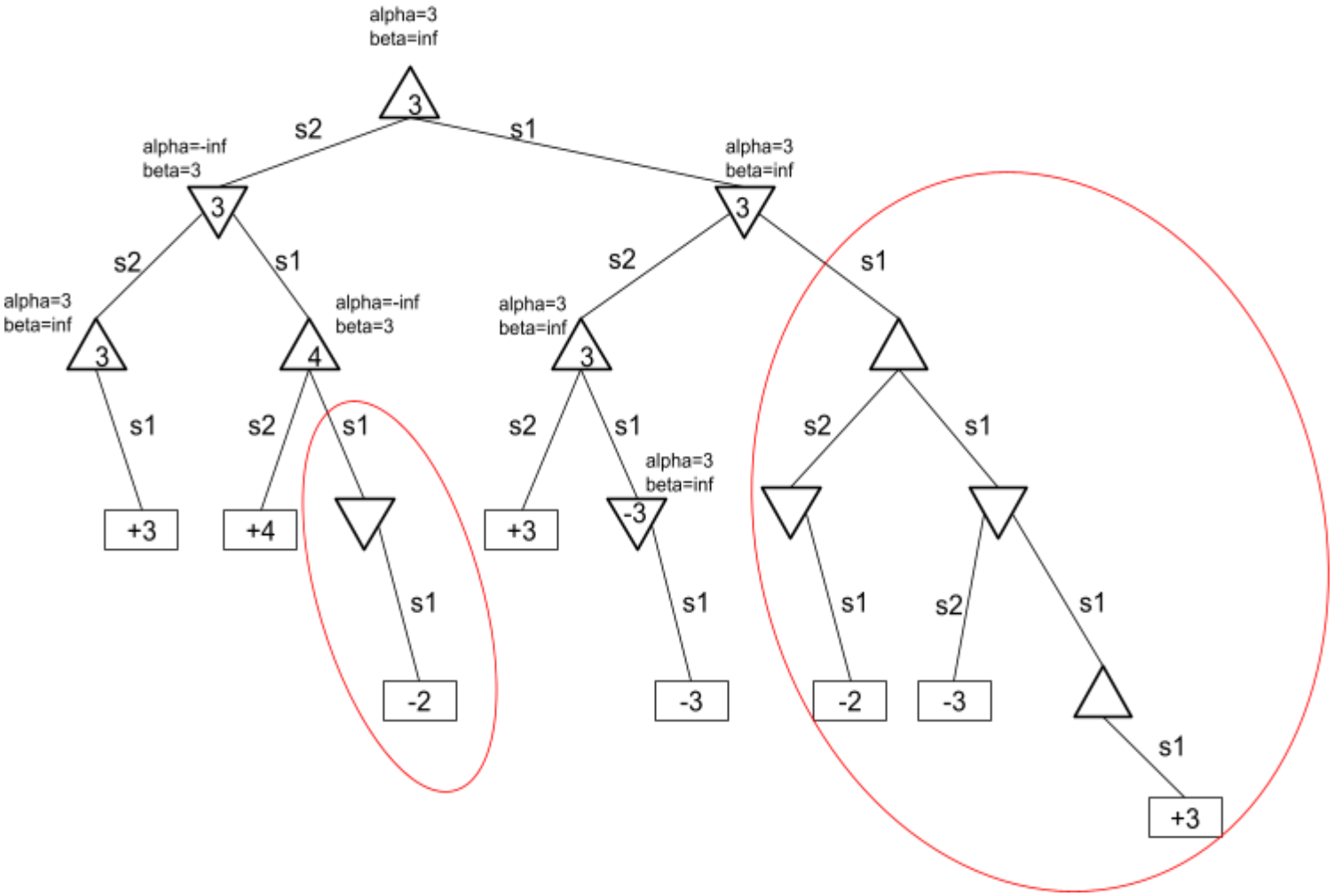
   For example, if the Max player pick 1 marble first, then the Min player chooses 2 marbles next, and finally the Min player chooses the last 2 marbles to win the game, the resulting terminal state has value +3. Instead, if the Max player chooses 1 marble first, the Min player chooses 1 marble next, the Max player then chooses 1 marble next, and finally the Min player chooses the last 2 marbles to win the game, then the resulting terminal state has value -3.

   Assume that the game has just started, and the Max player will make the first move, and then the players alternate. **All instructions must be followed, otherwise, marks will not be awarded.**

   a. Build a game tree for selecting the best move for the Max player. Don't include the invalid moves. At any node, **the child node created by s2 should be the left child, and the child node created by s1 should be the right child.** [3 Marks]

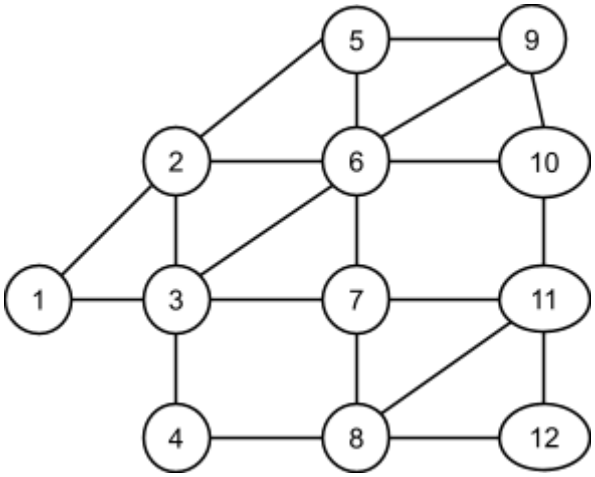


b. Perform alpha-beta pruning and show the final tree with final alpha-beta values, and **clearly mark the nodes that have been pruned by drawing an X beside those nodes in the tree**. Note: **When checking the children nodes, evaluate the left child before the right child**. [6 Marks]



c. What should be the first move by the Max player? [1 Mark]
Ans: Pick 2 marbles, if following minimax, Pick 1 or 2 marbles if following minimax with alpha beta pruning

4. Constraint Satisfaction Problem: Consider the following graph, having nodes 1 to 12. The nodes are to be colored with the colors R, G or B. Solve this using CSP, i.e., backtracking + forward checking + MRV. No two nodes connected by an edge should have the same color. In cases where 2 nodes are equally eligible to be colored next, follow the increasing order of the node number to decide which node to color next. While assigning a color to a node, follow the order R>G>B, and adapt accordingly as per the available colors for that node. When assigning a color to a node, add an X in the cells for the other colors in the row corresponding to that node in the table. Do the same thing if you are deleting a color from the domain of a node. Note that there should not be more than three red nodes. Fill the table shown below in your answer sheet. Stop at the first failure due to any assignment. The order column should mention the order in which the variables were assigned values in that table, even if that resulted in a failure. Start the assignment from node 1. **All instructions must be followed, otherwise, marks will not be awarded.** [5 Marks]



	R	G	B	Order
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				

	R	G	B	**Order**
1	R	X	X	1
2	X	G	X	2
3	X	X	B	3
4	X	G	X	10
5	X	X	B	5
6	R	X	X	4
7	X	G	X	6
8	X	X	B	11
9	X	G	X	7
10	X	X	B	8
11	R	X	X	9
12	X	G	X	12

5. Simplify the following propositional logic sentences to the simplest possible form that has only one proposition or True/False. Write "not possible" otherwise. Write all the equivalence "names" that you use. **All instructions must be followed, otherwise, marks will not be awarded.**

   a. $(A \Rightarrow \neg B \wedge \neg A) \Rightarrow ((A \wedge (B \vee A)) \wedge C)$ [2 Marks]

   Ans: A

$(\neg A \vee (\neg B \wedge \neg A)) \Rightarrow ((A \wedge (B \vee A)) \wedge C)$	[Implication Elimination]
$\neg(\neg A \vee (\neg B \wedge \neg A)) \vee ((A \wedge (B \vee A)) \wedge C)$	[Implication Elimination]
$\neg(\neg A) \vee ((A \wedge (B \vee A)) \wedge C)$	[Absorption]
$\neg(\neg A) \vee ((A) \wedge C)$	[Absorption]
$A \vee (A \wedge C)$	[DeMorgan's Law]
A	[Absorption]

   b. $(C \Rightarrow A) \wedge (A \Rightarrow B) \Rightarrow (C \Rightarrow B)$ [2 Marks]

   Ans: True

$(C \Rightarrow A) \wedge (A \Rightarrow B) \Rightarrow (C \Rightarrow B)$	
$(\neg C \vee A) \wedge (A \Rightarrow B) \Rightarrow (C \Rightarrow B)$	[Implication Elimination]
$(\neg C \vee A) \wedge (\neg A \vee B) \Rightarrow (C \Rightarrow B)$	[Implication Elimination]
$(\neg C \vee A) \wedge (\neg A \vee B) \Rightarrow (\neg C \vee B)$	[Implication Elimination]
$\neg((\neg C \vee A) \wedge (\neg A \vee B)) \vee (\neg C \vee B)$	[Implication Elimination]
$(\neg(\neg C \vee A) \vee \neg(\neg A \vee B)) \vee (\neg C \vee B)$	[DeMorgan's Law]
$((C \wedge \neg A) \vee (A \wedge \neg B)) \vee (\neg C \vee B)$	[DeMorgan's Law]
$(C \wedge \neg A) \vee \neg C \vee (A \wedge \neg B) \vee B$	[Commutativity]
$(C \vee \neg C) \wedge (\neg A \vee \neg C) \vee (A \wedge \neg B) \vee B$	[Distributing v over $\wedge$]
$T \wedge (\neg A \vee \neg C) \vee (A \vee B) \wedge (\neg B \vee B)$	[Distributing v over $\wedge$]
$T \wedge (\neg A \vee \neg C) \vee (A \vee B) \wedge T$	[Distributing v over $\wedge$]
$\neg A \vee A \vee \neg C \vee B$	[Commutativity]
$T \vee \neg C \vee B$	[Negation Law]
T	[Domination Law]

   c. $(((A \vee (A \wedge B)) \vee ((A \vee (A \wedge B)) \wedge B)) \vee (\neg A \wedge (\neg A \vee B)))$ [2 Marks]

   Ans: True

$(((A) \vee ((A \vee (A \wedge B)) \wedge B)) \vee (\neg A \wedge (\neg A \vee B)))$	[Absorption]
$(((A) \vee (A) \wedge B)) \vee (\neg A \wedge (\neg A \vee B)))$	[Absorption]
$(((A) \vee (A) \wedge B)) \vee (\neg A))$	[Absorption]
$(A) \vee (\neg A)$	[Absorption]
T	[Negation Law]

6. In the following, verify if the query can be derived from the knowledge base using Inference by Resolution Refutation only. Write all the equivalence "names" and inference rule "names" that you use. (Don't use truth values or statements like if A is True, then B will also be True). **All instructions must be followed, otherwise, marks will not be awarded.** [2 + 2 Marks]

   a. KB: $(\neg A \vee \neg C) \Rightarrow B$ Query: E
   $C \Rightarrow E$
   $\neg B$

   Augmented KB
   $(\neg A \vee \neg C) \Rightarrow B$
   $C \Rightarrow E$
   $\neg B$
   $\neg E$

Convert to Causal Form

¬(¬A ∨ ¬C) v B [Implication Elimination]
(A ∧ C) v B [DeMorgan's Law]
(A v B) ∧ (C v B) [Distributing v over ∧]
Two clauses
A v B
C v B

¬C v E [Implication Elimination]
¬B [Already in clausal form]
¬E [Already in clausal form]

Augment KB in clausal form

1. A v B
2. C v B
3. ¬C v E
4. ¬B
5. ¬E

Resolution in 2 an 4

6. C

Resolution in 6 and 3

7. E

Resolution in 5 and 7

8. False

Proved by Contradiction, E can be inferred from KB

b. KB: (A ∨ B) ⇔ (B ∨ C) Query: D
C⇒D

Augmented KB
(A ∨ B) ⇔ (B ∨ C)
C⇒D
¬D

Convert to Clausal Form
((A ∨ B) ⇒ (B ∨ C)) ∧ ((B ∨ C) ⇒ (A v B)) [Biconditional elimination]
(¬(A ∨ B) v (B ∨ C)) ∧ (¬(B ∨ C) v (A v B)) [Implication elimination]
((¬A ∧ ¬B) v (B v C)) ∧ ((¬B ∧ ¬C) v A v B) [Demorgan's Law]
((¬A v B)∧ (¬B v B) v C) ∧ ((¬B v B) ∧ (¬C v B) v A) [Distributing v over ∧]
((¬A v B) v C) ∧ ((¬C v B) v A) [Distributing v over ∧]
Two clauses
¬A v B v C
¬C v B v A

¬C v D [Implication Elimination]
¬D [Already in Clausal Form]

Augmented KB in Clausal Form

1. ¬A v B v C
2. ¬C v B v A
3. ¬C v D
4. ¬D

False cannot be derived so D cannot be inferred from the KB

————————END————————